

Documentation: SnakeGame

By Mahum Moazzam

Technical Decisions

- **Core logic: C# with zero Unity dependencies** (`SnakeGame.Core` namespace)
 - No `MonoBehaviour`, `Transform`, or `UnityEngine` references anywhere in this layer
 - Makes the rules directly unit testable without Unity's play mode or Test Framework runtime tricks
- **Presentation layer is a thin wrapper** (`SnakeGame.Presentation` namespace)
 - `MonoBehaviour` scripts only translate core logic state into GameObjects/UI — they contain no game rules themselves
- **Assembly Definitions** separate the two layers (`SnakeGame.Core.asmdef`, `SnakeGame.Core.Tests.asmdef`)
 - Enforces the separation at compile time, not just by convention
- **Dependency injection via constructors**
 - `SnakeGame` is built from `GameBoard`, `FoodSpawn`, and `SnakeBody` passed in, not created internally
 - Lets unit tests build small, controlled boards/snakes instead of a fixed real-game setup
- **`FoodSpawn` accepts an optional seeded `Random`**
 - Makes food-position tests deterministic instead of flaky

Design Patterns

- **Singleton: `GameManager`**
 - One authoritative game instance per scene
 - Multiple unrelated scripts (input, rendering, UI) all need a shared reference to it
- **Observer: `SnakeGame` events (`OnScoreChanged`, `OnGameOver`, `OnFoodSpawned`)**
 - Core logic never references UI or GameObjects directly
 - `UIManager`, `SnakeRend`, `FoodRend` react to events instead of being polled or called into
- **State: `GameState` enum (`Playing`, `Paused`, `GameOver`)**
 - `SnakeGame.Tick()` and `ChangeDirection()` branch on this instead of scattering boolean flags

Code Conventions

- PascalCase for classes, methods, and public properties
- camelCase with leading underscore for private fields (`_segmentInstances`, `_tickTimer`)
- One class per file, file name matches class name
- XML doc comments (`///`) on every public class and non-trivial method
- Namespaces split by layer: `SnakeGame.Core` vs `SnakeGame.Presentation`
- Naming is intentionally short/informal in a few places (`GridPos`, `FoodSpawn`, `SnakeRend`, `FoodRend`)

Known Bug Fixed During Development

- **Restart bug:** `SnakeRend` originally disabled itself (`enabled = false`) on game over to freeze the snake in place
 - A disabled `MonoBehaviour` never runs `Update()` again, so it could never detect that `GameManager` had created a new game after Restart
 - Game logic kept running correctly in the background (invisible), then died again a few seconds later, which looked like "Restart doesn't work"
 - Fix: replaced `enabled = false` with an internal `_isFrozen` flag, so `Update()` keeps running and can detect + resubscribe to a new `SnakeGame` instance on restart

AI Usage

AI (Claude) was used to resolve issues during development.

- Asked to tidy up initial class overview for `SnakeGame`
 - Prompt used: *This is a snapshot of my class overview for my snake game. Please give it appropriate formatting and reword any excessively long sentences into shorter ones for clarity.*
- Asked for help debugging
 - Prompt used: *The Restart button won't work correctly. Shows the previous game frozen after losing the level before Game Over canvas pops up again after a few seconds. Please identify the issue in my code.*
 - Prompt used: *none of my test scripts are showing up in Test Runner EditMode. Please let me know from the screenshot if I've appropriately placed scripts in the right folders.*
- Asked for documentation to be formatted in bullet-point form as I tend to ramble when writing/typing.

- Asked to tidy up haphazardly written code and to insert comments in the code to track mechanisms and ensure I can locate problem areas easier.
 - Prompt: *Here is my code. Please don't change anything, just tidy up whatever I give you into a readable format and add relevant comments to relevant sections.*